

Bitcoin Testnet4 交易、区块编码与 PoW 验证实验报告

小组成员：宋佳原、宋曰琦、杜昊霖

2026 年 7 月

目录

1	实验目的	3
2	实验环境与分工	3
3	实验原理	4
3.1	交易序列化	4
3.2	P2WPKH Script	4
3.3	区块、Merkle tree 与 PoW	4
4	Testnet4 交易构造与广播	5
4.1	未签名交易	5
4.2	找零与金额守恒	5
4.3	签名和 witness	6
4.4	预验证和广播	7
5	Raw Transaction 独立逐字节解析	8
6	已确认区块与 Coinbase 解析	10
6.1	区块选择	10
6.2	Coinbase transaction	11
7	Block、Merkle root 与 PoW 独立验证	13
8	实验结果讨论	15
9	结论	15
10	复现实验命令	15

目 录	2
A 核心程序	16
B 参考资料	16

1 实验目的

本实验在 Bitcoin Testnet4 上构造、签名并广播一笔原生 SegWit (P2WPKH) 交易，获取交易的 raw hex，并在不依赖 Bitcoin Core 解码接口的条件下按字节和比特解析 Transaction、Input、Output、锁定脚本和解锁数据。随后选取一个已确认 Testnet4 区块，解析其 80 字节区块头、全部交易与 Coinbase transaction，独立重算 txid、wtxid、Merkle root、witness commitment 和区块哈希，最终验证工作量证明 (Proof of Work, PoW)。

2 实验环境与分工

项目	配置
操作系统	Ubuntu 20.04 (Windows/WSL 环境)
Bitcoin Core	31.1, 网络参数 <code>-testnet4</code>
节点模式	裁剪节点, <code>prune=1000</code>
辅助工具	<code>bitcoin-cli</code> 、Python 3、 <code>jq</code> 、 <code>xxd</code>
报告编译	Windows TeX Live/MiKTeX, XeLaTeX

表 1: 实验环境

小组共同完成节点部署、交易构造、二进制解析和结果复核。宋佳原负责节点操作与交易广播，宋曰琦负责交易及 Script 字段整理，杜昊霖负责区块、Merkle tree 与 PoW 验证；三人共同完成报告审阅。

节点同步结果见图 1。节点处于 Testnet4，区块与区块头高度一致，`verificationprogress` 为 1，且 `initialblockdownload` 为 false，说明初始同步完成。

```
'sly@sjy:~/bitcoin-tx-lab$ bitcoin-cli -testnet4 getblockchaininfo |
> jq '{chain,blocks,headers,verificationprogress,initialblockdownload,pruned}'
{
  "chain": "testnet4",
  "blocks": 145317,
  "headers": 145317,
  "verificationprogress": 1,
  "initialblockdownload": false,
  "pruned": true
}
```

(a) Testnet4 节点同步完成

```
{
  "txid": "66885f3456d3202060e56f77b4fa4d299e61aca495fa54913c7733f02809c45d",
  "vout": 0,
  "address": "tb1qw0s94yguv2gdxagftpd74dpr6smx6tcx80nrzq",
  "scriptPubKey": "001471e05a911c6290d37749585beab423d4366d2f06",
  "amount": 0.00333681,
  "confirmations": 0,
  "spendable": true,
  "safe": false
}
```

(b) Faucet UTXO 信息

图 1: 节点状态与实验初始 UTXO

3 实验原理

3.1 交易序列化

SegWit 交易采用如下网络序列化:

$$\begin{aligned} &\text{version} \parallel \text{marker} \parallel \text{flag} \parallel \text{vin count} \parallel \text{inputs} \\ &\parallel \text{vout count} \parallel \text{outputs} \parallel \text{witnesses} \parallel \text{locktime}. \end{aligned}$$

固定宽度整数使用小端编码,数组长度和脚本长度使用 CompactSize。对于原生 P2WPKH, `scriptSig` 为空, 签名与公钥位于 witness 栈。txid 排除 marker、flag 和 witness; wtxid 对完整交易序列化做两次 SHA-256:

$$\text{txid} = \text{rev}(\text{SHA256d}(T_{\text{stripped}})), \quad \text{wtxid} = \text{rev}(\text{SHA256d}(T_{\text{full}})).$$

3.2 P2WPKH Script

本实验两个输出的锁定脚本均为

$$00 \ 14 \ < 20\text{-byte HASH160}(\text{pubkey}) \ >,$$

其中 00 是 witness version 0, 14 表示后续 program 长 20 字节。解锁时 witness 栈依次压入 DER-ECDSA 签名和 33 字节压缩公钥。传统语境中的 unlocking script 是 `scriptSig`; 在本实验原生 SegWit 输入中它必须为空, 解锁数据已迁移到 witness。

3.3 区块、Merkle tree 与 PoW

区块由 80 字节头部、CompactSize 交易数和交易列表组成。区块头字段见表 2。

字节偏移	比特偏移	长度/字节	字段	编码
0	0-31	4	version	小端
4	32-287	32	previous block hash	内部字节序
36	288-543	32	Merkle root	内部字节序
68	544-575	4	timestamp	小端
72	576-607	4	nBits	compact target, 小端
76	608-639	4	nonce	小端

表 2: 80 字节区块头格式

交易 txid 作为叶子逐层两两拼接并做 SHA256d; 奇数层复制最后一个结点, 最终得到 Merkle root。PoW 将区块头做 SHA256d, 并验证

$$H_{\text{integer}} \leq \text{target}, \quad \text{target} = c \cdot 256^{e-3},$$

其中 c 和 e 由 nBits 的 coefficient 与 exponent 给出。

4 Testnet4 交易构造与广播

4.1 未签名交易

Faucet UTXO 为 333681 sat, 输入 outpoint 为:

66885f3456d3202060e56f77b4fa4d299e61aca495fa54913c7733f02809c45d:0

目标地址为:

tb1qheuvew7svumewwn4nvwu7h3nzn9cmysnhhs3

发送金额为 10000 sat。使用 `createrawtransaction` 后得到一个输入和一个输出, 输入 `scriptSig` 为空, `sequence` 为 $4294967293 = 0xffffffff$, 支持 BIP125 RBF。未签名交易的 Core 解码见图 2。

```

{
  "txid": "e6c5cb745e1eb8055c12713a0d79a4a36972e1b599e62d943a963cb8ea9a4dee",
  "version": 2,
  "locktime": 0,
  "vin": [
    {
      "txid": "66885f3456d3202060e56f77b4fa4d299e61aca495fa54913c7733f02809c45d",
      "vout": 0,
      "scriptSig": {
        "asm": "",
        "hex": ""
      },
      "sequence": 4294967293
    }
  ],
  "vout": [
    {
      "value": 0.0001,
      "n": 0,
      "scriptPubKey": {
        "asm": "0 be78ccbbd06737973a759b99cf5e3314c65c6c90",
        "desc": "addr(tb1qheuvew7svumewwn4nvwu7h3nzn9cmysnhhs3)#na46mvp6",
        "hex": "0014be78ccbbd06737973a759b99cf5e3314c65c6c90",
        "address": "tb1qheuvew7svumewwn4nvwu7h3nzn9cmysnhhs3",
        "type": "witness_v0_keyhash"
      }
    }
  ]
}

```

(a) 未签名交易

(b) 手续费与找零位置

图 2: 交易构造与资金补全

4.2 找零与金额守恒

调用 `fundrawtransaction`, 指定 2 sat/vB 并允许使用尚未确认的 faucet 输入。钱包添加 323260 sat 找零, 最终金额关系为

$$333681 = 323260 + 10000 + 421 \quad (\text{sat}).$$

构造时父交易尚未确认, Bitcoin Core 会兼顾父子交易包的目标费率, 因此子交易最终手续费为 421 sat。广播前父交易已确认, 故子交易自身费率为 $421/141 \approx 2.986$ sat/vB。两个输出及守恒检查见图 3。



图 3: 交易输出与手续费

```

sjy@sjy:~/bitcoin-tx-lab$ echo "Input:      333681 sat"
Input:      333681 sat
sjy@sjy:~/bitcoin-tx-lab$ echo "Change:     323260 sat"
Change:     323260 sat
sjy@sjy:~/bitcoin-tx-lab$ echo "Destination: 10000 sat"
Destination: 10000 sat
sjy@sjy:~/bitcoin-tx-lab$ echo "Child fee:   421 sat"
Child fee:   421 sat
sjy@sjy:~/bitcoin-tx-lab$ echo "Check: $((323260 + 10000 + 421)) sat"
Check: 333681 sat

```

图 4: 输入、找零、目标输出与手续费的金额守恒

4.3 签名和 witness

钱包使用对应私钥签名后, `complete=true`。最终交易长 222 字节, `vsize` 为 141, `weight` 为 561。txid 与 wtxid 分别为:

```
dea08b1b3e8595314a7affd5e0299b81537c48eb78b8634d317f0c4f44011d5f
```

```
df9cc5342bca17c969b665ae32f133dd1d0d86b6b8e09fa0c4ce41898f416a29
```

witness 包含 71 字节 DER-ECDSA 签名 (末尾 01, 即 SIGHASH_ALL) 和 33 字节压缩公钥。由于签名只写入 witness, funded transaction 与签名后交易的 txid 相同, 而 wtxid 不同。

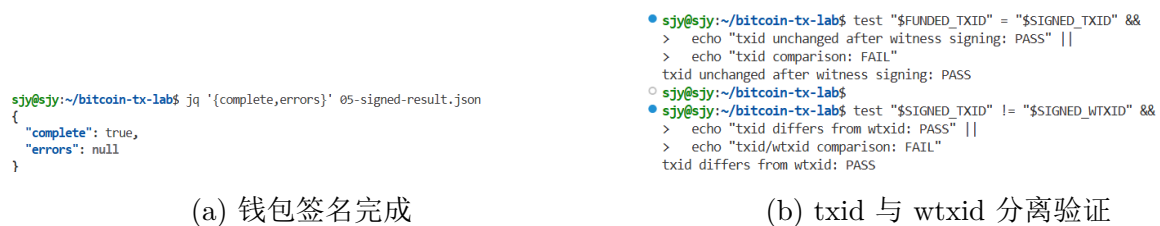


图 5: SegWit 交易签名结果

```

• sjy@sjy:~/bitcoin-tx-lab$ jq '{
> witness_item_count: (.vin[0].txinwitness | length),
> signature_bytes: (.vin[0].txinwitness[0] | length / 2),
> signature_last_byte: (.vin[0].txinwitness[0][-2:]),
> public_key_bytes: (.vin[0].txinwitness[1] | length / 2),
> public_key_prefix: (.vin[0].txinwitness[1][0:2]),
> scriptSig_hex: .vin[0].scriptSig.hex
> }' 05-signed.json
{
  "witness_item_count": 2,
  "signature_bytes": 71,
  "signature_last_byte": "01",
  "public_key_bytes": 33,
  "public_key_prefix": "02",
  "scriptSig_hex": ""
}

```

(a) P2WPKH witness 栈

```

"outputs": [
  {
    "n": 0,
    "value": 0.0032326,
    "value_sats": 323260,
    "script_asm": "0 4db668b024f3b4e14fbb8de2a043520b505ceea7",
    "script_hex": "00144db668b024f3b4e14fbb8de2a043520b505ceea7",
    "address": "tb1qfkmx3vpy7w6wznm3h32qs6jpdg9em484qqv7s",
    "type": "witness_v0_keyhash"
  },
  {
    "n": 1,
    "value": 0.0001,
    "value_sats": 10000,
    "script_asm": "0 be78ccbbd06737973a759b99cf5e3314c65c6c90",
    "script_hex": "0014be78ccbbd06737973a759b99cf5e3314c65c6c90",
    "address": "tb1qheuvew7svumewm4mwvu7h3nzn9cmysnhsv3",
    "type": "witness_v0_keyhash"
  }
]

```

(b) 两个 P2WPKH 输出脚本

图 6: 解锁数据与锁定脚本

4.4 预验证和广播

广播前 `testmempoolaccept` 返回 `allowed=true`; 随后 `sendrawtransaction` 返回的 `txid` 与本地解码值完全一致。mempool 中 `unbroadcast=false`, 表明交易已经传播, 且 `depends=[]` 表明父交易已经确认。验证结果和公共浏览器记录见图 7、图 8。

```

sjy@sjy:~/bitcoin-tx-lab$ jq '{
> txid,
> wtxid,
> allowed,
> vsize,
> fees,
> reject_reason: .reject-reason
> }' 06-mempool-check.json
{
  "txid": "dea08b1b3e8595314a7affd5e0299b81537c48eb78b8634d317f0c4f44011d5f",
  "wtxid": "df9cc5342bca17c969b665ae32f133dd1dd8b6b8e09fa0c4ce41898f416a29",
  "allowed": true,
  "vsize": 141,
  "fees": {
    "base": 4.21e-06,
    "effective-feerate": 2.985e-05,
    "effective-includes": [
      "df9cc5342bca17c969b665ae32f133dd1dd8b6b8e09fa0c4ce41898f416a29"
    ]
  },
  "reject_reason": null
}

```

(a) mempool 预验证通过

```

• sjy@sjy:~/bitcoin-tx-lab$ test "$EXPECTED_TXID" = "$TXID" &&
> echo "Broadcast txid verification: PASS" ||
> echo "Broadcast txid verification: FAIL"
Broadcast txid verification: PASS

```

(b) 广播 txid 一致性验证

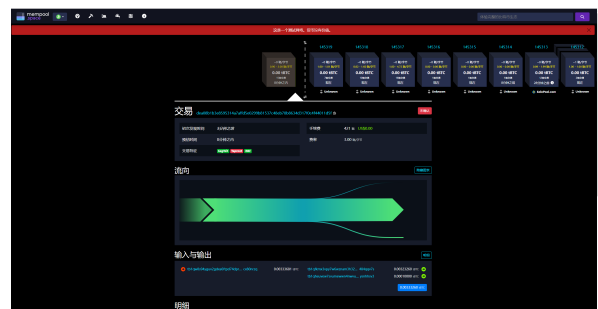
图 7: 交易预验证与广播

```

{
  "vsize": 141,
  "weight": 561,
  "time": 1784773013,
  "height": 145318,
  "ancestorcount": 1,
  "ancestorsize": 141,
  "fees": {
    "base": 4.21e-06,
    "modified": 4.21e-06,
    "ancestor": 4.21e-06,
    "descendant": 4.21e-06,
    "chunk": 4.21e-06
  },
  "depends": [],
  "spentby": [],
  "bip125_replaceable": true,
  "unbroadcast": false
}

```

(a) 广播后的 mempool 状态



(b) 公共 Testnet4 浏览器记录

图 8: 交易网络传播证据

实验时间窗口内该交易仍为未确认状态。由于实验要求的交易格式、签名和广播均可在 mempool 阶段验证，而 Testnet4 的出块和矿工选包具有不确定性，后续区块实验选用另一个已确认 Testnet4 区块。报告不将该区块描述为实验交易所在区块。

5 Raw Transaction 独立逐字节解析

编写 `parse_tx.py`，直接读取 `05-signed.hex`，维护字节游标并输出 byte range、bit range、字段长度、hex、完整 binary 和解释。解析器独立重建 stripped serialization 和 full serialization，重算 txid 与 wtxid。

实验交易的主要字段见表 3。

表 3: 实验交易的逐字段编码

字节偏移	长度	字段	解析结果
0-3	4	version	2, 小端编码
4	1	marker	00
5	1	flag	01, 存在 witness
6	1	input count	1
7-38	32	previous txid	反转后为 faucet txid
39-42	4	previous vout	0
43	1	scriptSig length	0
44-47	4	sequence	fdffffff, 即 0xffffffffd
48	1	output count	2
后续	可变	outputs	323260 sat 找零与 10000 sat 目标输出
后续	可变	witness	71 字节签名、33 字节压缩公钥
末尾 4 字节	4	locktime	0

逐字节与逐比特输出见图 9 至图 11。


```

sly@sjy:~/bitcoin-tx-lab$ python3 parse_tx.py 05-signed.hex \
> --expected-txid dea08b1b3e8595314a7affd5e0299b81537c48eb78b8634d317f0c4f44011d5f \
> --expected-wtxid df9cc5342bca17c969b665ae32f133dd1d0d86b6b8e09fa0c4ce41898f416a29 \
> --prefix 12-experiment-tx
{
  "bytes": 222,
  "parsed_bytes": 222,
  "segwit": true,
  "version": 2,
  "input_count": 1,
  "output_count": 2,
  "locktime": 0,
  "txid": "dea08b1b3e8595314a7affd5e0299b81537c48eb78b8634d317f0c4f44011d5f",
  "wtxid": "df9cc5342bca17c969b665ae32f133dd1d0d86b6b8e09fa0c4ce41898f416a29",
  "is_coinbase": false,
  "coinbase_height": null
}
Parsed bytes: 222/222: PASS
TXID verification: PASS
WTXID verification: PASS
Byte table: 12-experiment-tx-bytes.tsv
JSON result: 12-experiment-tx-parsed.json

```

图 9: 独立解析 222 字节交易并验证 txid、wtxid

BYTE	BIT	FIELD	HEX	DECODED	
0-3	0-31	version	02000000	2	
4-4	32-39	marker	00	Segid marker	
5-5	40-47	flag	01	witness present	
6-6	48-55	input_count	01	1	
7-38	56-311	input[0].previous_txid	5dc40928f03773c9154fa...		
39-42	312-343	input[0].previous_vout	00000000		
43-43	344-351	input[0].scriptsig_length	00	0	
44-43	352-351	input[0].scriptsig		empty (native Segid unlock is in witness)	
44-47	352-383	input[0].sequence	ffffffff		
48-48	384-391	output_count	02	2	
49-56	392-455	output[0].value	bce0d40000000000	323260 sat	
57-57	456-463	output[0].scriptpubkey_length	16	22	
58-79	464-639	output[0].scriptpubkey	08144d668a02743bf4ef...	P2A0P: OP_0 PUSH0 4d668a02743bf4	
e14f8b8e2a045208595ceea7					
80-87	640-703	output[1].value	1027000000000000	10000 sat	
88-88	704-711	output[1].scriptpubkey_length	16	22	
89-110	712-887	output[1].scriptpubkey	0814be78ccbb06737973a...	P2A0P: OP_0 PUSH0 be78ccbb06737	
973a759b99c5e3314c65c690					
111-111	888-895	witness[0].item_count	02	2	
112-183	896-903	witness[0].item[0].length	47	71	
113-183	904-1471	witness[0].item[0]		gprature: s1ghash=0d1	
184-184	1472-1479	witness[0].item[1].length	21	33	
185-217	1480-1473	witness[0].item[1]		blt key; prefix=0d02	
218-217	1474-1475	locktime		00000000	0

(a) 交易字段 byte/bit offset

(b) witness 与 locktime 字段

图 10: 实验交易的逐字段解析

```

• sjy@sjy:~/bitcoin-tx-lab$ head -n 7 12-experiment-tx-bytes.tsv |
> column -t -s $'\t'
byte_range  bit_range  length  field                hex
      binary
                                decoded
0-3          0-31        4        version            02000000
00000010 00000000 00000000 00000000

4-4          32-39        1        marker              00
00000000

5-5          40-47        1        flag                01
00000001
Segwit marker

6-6          48-55        1        input_count          01
00000001
witness present

7-38        56-311       32        input[0].previous_txid 5dc40928f033773c9154fa95a4ac619e294dfab4776fe5602020d3563451
866 01011101 11000100 00001001 00101000 11110000 00110011 01110111 00111100 10010001 01010100 11111010 10010101 10010100 10101100 01100001 10011110 00101001 01001101 11111010 10110100 01110111 01101111 11100101 01100000 00100000 00100000 11010011 01010110 00110100 01011111 10001000 01100110
39-42        312-343       4        input[0].previous_vout 00000000
00000000 00000000 00000000 00000000

```

图 11: 字段的逐比特二进制表示

解析器恰好消费 222/222 字节, txid 和 wtxid 均与 Bitcoin Core 结果相同, 说明字段边界、CompactSize、端序处理以及 witness 剥离过程正确。完整逐 bit 结果保存在 12-experiment-tx-bytes.tsv 的 binary 列。

6 已确认区块与 Coinbase 解析

6.1 区块选择

选取高度 145319 的已确认区块:

```
0000000000012057bbfe9e5ce7979a8fe06c37ac350808187a97bce8635404bf
```

该区块共 8 笔交易, raw block 长 2083 字节, stripped size 为 1070, weight 为 5293, 适合完整逐笔解析。选择、长度和区块头信息见图 12。

```

• sjy@sjy:~/bitcoin-tx-lab$ test "$BLOCK_HASH" = "$CORE_BLOCK_HASH" &&
> echo "Block selection: PASS" ||
> echo "Block selection: FAIL"
Block selection: PASS
• sjy@sjy:~/bitcoin-tx-lab$

```

(a) 区块高度与哈希对应

```

sjy@sjy:~/bitcoin-tx-lab$ printf 'Block hash: %s\n' "$BLOCK_HASH"
Block hash: 0000000000012057bbfe9e5ce7979a8fe06c37ac350808187a97bce8635404bf
sjy@sjy:~/bitcoin-tx-lab$ printf 'Block hex format: PASS\n'
Block hex format: PASS
sjy@sjy:~/bitcoin-tx-lab$ printf 'Raw block: %d bytes\n' "${#BLOCK_HEX} / 2 )"
Raw block: 2083 bytes
sjy@sjy:~/bitcoin-tx-lab$ printf 'Block header: %d bytes\n' "${#HEADER_HEX} / 2 )"
Block header: 80 bytes

```

(b) 区块及区块头长度

图 12: 已确认 Testnet4 区块选择

```
> }' 10-block-summary.json
{
  "hash": "000000000012057bbfe9e5ce7979a8fe06c37ac350808187a97bcce8635404bf",
  "height": 145319,
  "confirmations": 1,
  "version": 849559552,
  "versionHex": "32a34000",
  "merkleroot": "af20a4e3aa86a46c636d2ee2631b69a3fe5d677eb5a1135d8b103e97320c2726",
  "time": 1784780486,
  "mediantime": 1784775248,
  "nonce": 3190817204,
  "bits": "1d00ffff",
  "difficulty": 1,
  "nTx": 8,
  "size": 2083,
  "strippedsize": 1070,
  "weight": 5293,
  "previousblockhash": "000000000b4ebcfb04e59fb8125e0c26aebc2af0dc850c55bb4d0e7b47d1465"
}
```

图 13: 区块头、交易数与规模信息

```
● sjy@sjy:~/bitcoin-tx-lab$ test "$EXPECTED_COUNT" -eq "$ACTUAL_COUNT" &&
> echo "Transaction count: PASS" ||
> echo "Transaction count: FAIL"
Transaction count: PASS
```

图 14: 区块声明的交易数与实际解析数量一致

6.2 Coinbase transaction

区块位置 0 的交易为 Coinbase, 其 txid 与 wtxid 为:

```
2526d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473
2202b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939
```

该交易长 188 字节。输入 outpoint 在原始序列化中是 32 字节全零 txid 和 ffffffff vout; 其 scriptSig 为 03a7370200040776616a0c0dc0446a0000000000000000。首个 push 的 3 字节 a73702 以小端解释为 145319, 符合 BIP34 区块高度。

Coinbase 的 sequence 为 0xffffffff, witness reserved value 为 32 字节全零。它有两个输出: 第一个向 P2WPKH 地址支付 50.00002049 tBTC; 第二个是 0 金额 OP_RETURN, 格式为

6a 24 aa21a9ed < 32-byte commitment >.

Coinbase 输入、输出金额和 commitment 见图 15。

```

• sjy@sjy:~/bitcoin-tx-lab$ jq '{
  > coinbase_txid:.txid,
  > coinbase_wtxid:.hash,
  > coinbase_script:.vin[0].coinbase,
  > coinbase_script_bytes:(.vin[0].coinbase | length / 2),
  > sequence:.vin[0].sequence,
  > witness:.vin[0].txinwitness
  > }' 11-coinbase.json
{
  "coinbase_txid": "2526d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473",
  "coinbase_wtxid": "2202b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939",
  "coinbase_script": "03a7370200040776616a0c0dc0446a00000000000000",
  "coinbase_script_bytes": 23,
  "sequence": 4294967295,
  "witness": [
    "0000000000000000000000000000000000000000000000000000000000000000"
  ]
}

```

图 15: Coinbase transaction 的特殊输入结构

```

• sjy@sjy:~/bitcoin-tx-lab$ jq '{
  > total_btc:(.vout[] | value | add),
  > total_sats:(.vout[] | value * 100000000 | round) | add)
  > }' 11-coinbase.json
{
  "total_btc": 50.00002049,
  "total_sats": 5000002049
}

```

(a) Coinbase 总输出

```

{
  "n": 1,
  "value": 0,
  "asm": "OP_RETURN aa21a9ed36e61fb7af98bde2bca631f60d4499d42dbd77f27779a2b1f3201041e5d0246",
  "hex": "6a24aa21a9ed36e61fb7af98bde2bca631f60d4499d42dbd77f27779a2b1f3201041e5d0246",
  "type": "nulldata"
}

```

```

• sjy@sjy:~/bitcoin-tx-lab$ witness_commitment=$(jq -r '
  > .vout[] |
  > select(.scriptPubkey.hex | startswith("6a24aa21a9ed")) |
  > .scriptPubkey.hex[12:76]
  > ' 11-coinbase.json)
• sjy@sjy:~/bitcoin-tx-lab$ printf 'witness commitment: %s\n' "$witness_commitment"
witness commitment: b36e61fb7af98bde2bca631f60d4499d42dbd77f27779a2b1f3201041e5d0246
• sjy@sjy:~/bitcoin-tx-lab$ printf 'commitment length: %d bytes\n' \
  > "$(( ${#witness_commitment} / 2 ))"
commitment length: 32 bytes

```

(b) SegWit witness commitment

图 16: Coinbase 奖励与 witness 承诺

```

• sjy@sjy:~/bitcoin-tx-lab$ xxd -r -p 11-coinbase.hex |
  > xxd -g 1 |
  > tee 11-coinbase-hexdump.txt
00000000: 01 00 00 00 00 01 01 00 00 00 00 00 00 00 00 00 .....
00000010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00000020: 00 00 00 00 00 00 ff ff ff ff 17 03 a7 37 02 .....7.
00000030: 00 04 07 76 61 6a 0c 0d c0 44 6a 00 00 00 00 00 ...vaj...Dj....
00000040: 00 00 00 ff ff ff ff 02 01 fa 05 2a 01 00 00 00 .....*...
00000050: 16 00 14 85 e9 d1 6e fc d6 18 14 f8 4f d2 c8 e9 .....n....O...
00000060: 96 76 8b a4 3f c3 1c 00 00 00 00 00 00 00 26 .....?.....&
00000070: 6a 24 aa 21 a9 ed b3 6e 61 fb 7a f9 8b de 2b ca j$.l...na.z...+
00000080: 63 1f 60 d4 49 9d 42 db d7 7f 27 77 9a 2b 1f 32 c...I.B...w.+2
00000090: 01 04 1e 5d 02 46 01 20 00 00 00 00 00 00 00 ...].F.....
000000a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000000b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

(a) Coinbase raw hex hexdump

```

{
  "position": 0,
  "txid": "2526d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473",
  "wtxid": "2202b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939",
  "size": 188,
  "vsize": 161,
  "weight": 644,
  "coinbase": true
},
{
  "position": 1,
  "txid": "ce77c7924078ecff60f6a5fed4108f403d8de3879dd2a791917a8f19f3bf1a7",
  "wtxid": "d8eb208bde13438e30f94d290408f4ea624c1123956480e1193f0b3de097b9ab",
  "size": 222,
  "vsize": 141,
  "weight": 561,
  "coinbase": false
},
{
  "position": 2,
  "txid": "8c2afa41d1086d0c826c4dc93c141830d3591468a0594ba652d634d7082a189",
  "wtxid": "c59d39315546b32fee5570b657d26be260767005c729809394e835621a5d707",
  "size": 222,
  "vsize": 141,
  "weight": 561,
  "coinbase": false
}

```

(b) 区块中的 8 笔交易列表

图 17: Coinbase 原始编码与区块交易索引

同一 `parse_tx.py` 对 Coinbase raw hex 解析出 188/188 字节, 自动识别 `is_coinbase` 为 true, 并解析出高度 145319, 同时重算 txid、wtxid。结果见图 18。

```

sly@sly:~/bitcoin-tx-lab$
thon3 parse_tx.py 11-coinbase.hex \
> --expected-txid 2526d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473 \
> --expected-wtxid 2202b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939 \
> --prefix 13-coinbase
{
  "bytes": 188,
  "parsed_bytes": 188,
  "segwit": true,
  "version": 1,
  "input_count": 1,
  "output_count": 2,
  "locktime": 0,
  "txid": "2526d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473",
  "wtxid": "2202b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939",
  "is_coinbase": true,
  "coinbase_height": 145319
}
Parsed bytes: 188/188: PASS
TXID verification: PASS
WTXID verification: PASS
Byte table: 13-coinbase-bytes.tsv
JSON result: 13-coinbase-parsed.json

```

图 18: Coinbase 独立解析与哈希验证

```

{
  "summary": {
    "bytes": 188,
    "parsed_bytes": 188,
    "segwit": true,
    "version": 1,
    "input_count": 1,
    "output_count": 2,
    "locktime": 0,
    "txid": "2526d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473",
    "wtxid": "2202b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939",
    "is_coinbase": true,
    "coinbase_height": 145319
  },
  "input": {
    "previous_txid": "0000000000000000000000000000000000000000000000000000000000000000",
    "previous_vout": 4294967295,
    "scriptSig": "03a7370200040776616a0c0dc0446a000000000000000",
    "sequence": 4294967295
  },
  "witness": [
    "0000000000000000000000000000000000000000000000000000000000000000"
  ]
}

```

图 19: Coinbase outpoint、scriptSig 与 witness 结果

7 Block、Merkle root 与 PoW 独立验证

编写 `parse_block.py`, 从 `10-block.hex` 的偏移 0 开始读取 80 字节区块头, 再解析 CompactSize 交易数并循环调用交易边界解析器。程序得到 8 笔交易, 最终游标为 2083, 与 raw block 长度相同。

区块头关键结果如下:

字段	值
version	849559552 (32a34000)
previous block hash	0000000000b4ebcf...0e7b47d1465
Merkle root	af20a4e3aa86a46c...8b103e97320c2726
timestamp	1784780486
nBits	1d00ffff
nonce	3190817204
target	00000000ffff0000...0000000000000000
block hash	000000000012057b...a97bcce8635404bf

表 4: 独立解析的区块头结果

由 1d00ffff 得 $e = 0x1d$ 、 $c = 0x00ffff$, 从而

$$T = 00000000ffff0000...0000.$$

区块头 SHA256d 对应整数为

$$H = 000000000012057bbfe9e5ce...a97bcce8635404bf,$$

显然 $H < T$, 故 PoW 有效。

程序以所有交易的内部字节序 txid 构造 Merkle tree, 计算结果为:

```
af20a4e3aa86a46c636d2ee2631b69a3fe5d677eb5a1135d8b103e97320c2726
```

它与头部完全一致。对 witness tree, 将 Coinbase wtxid 叶子置零, 得到 witness Merkle root:

```
b9233c8d1e679864d92b9754262dfd1239c36a9f1c4b9389980e37f50f20d467
```

再与 32 字节 reserved value 拼接做 SHA256d, 得到:

```
b36e61fb7af98bde2bca631f60d4499d42dbd77f27779a2b1f3201041e5d0246
```

与 Coinbase OP_RETURN 中存储的 commitment 一致。

Listing 1: 区块独立验证结果

```
All Block Bytes Consumed: PASS
Transaction Count: PASS
Block Hash: PASS
Proof Of Work: PASS
Merkle Root: PASS
Witness Commitment: PASS
Coinbase First: PASS
```

上述结果来自 14-block-verification.txt, 而非 Bitcoin Core 对同一字段的直接复述。Core JSON 仅作为交叉对照。至此, Block、Transaction、Coinbase、Input、Output、scriptPubKey、scriptSig/witness 和 PoW 均获得了独立解析或验证。

8 实验结果讨论

1. **字节序是最主要的实现陷阱。**数值字段采用小端，而 RPC 显示的 txid、block hash、Merkle root 通常为内部哈希字节反转后的阅读顺序。
2. **txid 与 wtxid 承诺范围不同。**本实验签名写入 witness 后 txid 不变，而 wtxid 改变，实际结果验证了 SegWit 的隔离设计。
3. **scriptSig 为空不代表没有解锁数据。**原生 P2WPKH 使用 witness 中的签名和公钥完成解锁；Coinbase 虽有 scriptSig，却不引用普通 UTXO。
4. **未确认不影响序列化研究。**自建交易已通过节点规则、广播并在公共浏览器可见；确认只影响其是否被某个区块承诺。区块和 PoW 部分使用独立的已确认 Testnet4 区块，二者研究对象和结论边界在报告中明确区分。
5. **PoW 验证不仅是观察前导零。**程序从 nBits 恢复 256 位 target，将 SHA256d 结果按共识所需整数解释并做严格不等式比较。

9 结论

本实验成功在 Bitcoin Testnet4 上构造、签名并广播一笔 P2WPKH 交易，保存其 222 字节 raw hex，并逐字段、逐字节和逐 bit 解析。独立程序重算的 txid 与 wtxid 均与 Bitcoin Core 相同。实验进一步解析高度 145319 的完整区块及 Coinbase，验证 8 笔交易边界、BIP34 高度、奖励输出和 SegWit commitment。最后从 80 字节区块头重算区块哈希、Merkle root、target 与 PoW，全部检查通过。实验由此完整建立了从 UTXO、Script、交易序列化到区块承诺和工作量证明的层次化理解。

10 复现实验命令

```
# 交易逐字节解析
python3 parse_tx.py 05-signed.hex \
  --expected-txid
  dea08b1b3e8595314a7affd5e0299b81537c48eb78b8634d317f0c4f44011d5f \
  --expected-wtxid
  df9cc5342bca17c969b665ae32f133dd1d0d86b6b8e09fa0c4ce41898f416a29 \
  --prefix 12-experiment-tx

# Coinbase 逐字节解析
python3 parse_tx.py 11-coinbase.hex \
```

```

--expected-txid 2526
    d3c43f26d36cb7683e0f3c5c369ff2819cd3316f70f1d893b26159338473 \
--expected-wtxid 2202
    b18db31e3b6d9d16ad9afc8a349f250205e9b601261a8f762c49b3ab3939 \
--prefix 13-coinbase

# 完整区块、Merkle root、witness commitment 和 PoW
python3 parse_block.py 10-block.hex \
--expected-hash 000000000012057
    bbfe9e5ce7979a8fe06c37ac350808187a97bcce8635404bf \
--prefix 14-block

```

A 核心程序

完整源代码随报告提交。交易解析程序为 `parse_tx.py`，区块验证程序为 `parse_block.py`。二者只使用 Python 标准库。为控制报告篇幅，不在正文重复打印全部逐 bit TSV；12-experiment-tx-by-tsv、13-coinbase-bytes.tsv 和 14-block-header-bytes.tsv 作为实验附件提交。

B 参考资料

1. Bitcoin Developer Reference, Transactions: <https://developer.bitcoin.org/reference/transactions.html>。
2. Bitcoin Developer Reference, Block Chain: https://developer.bitcoin.org/reference/block_chain.html。
3. BIP 34, Block v2, Height in Coinbase: <https://bips.dev/34/>。
4. BIP 141, Segregated Witness: <https://bips.dev/141/>。
5. BIP 143, Transaction Signature Verification for Version 0 Witness Program: <https://bips.dev/143/>。
6. BIP 94, Testnet 4: <https://bips.dev/94/>。